

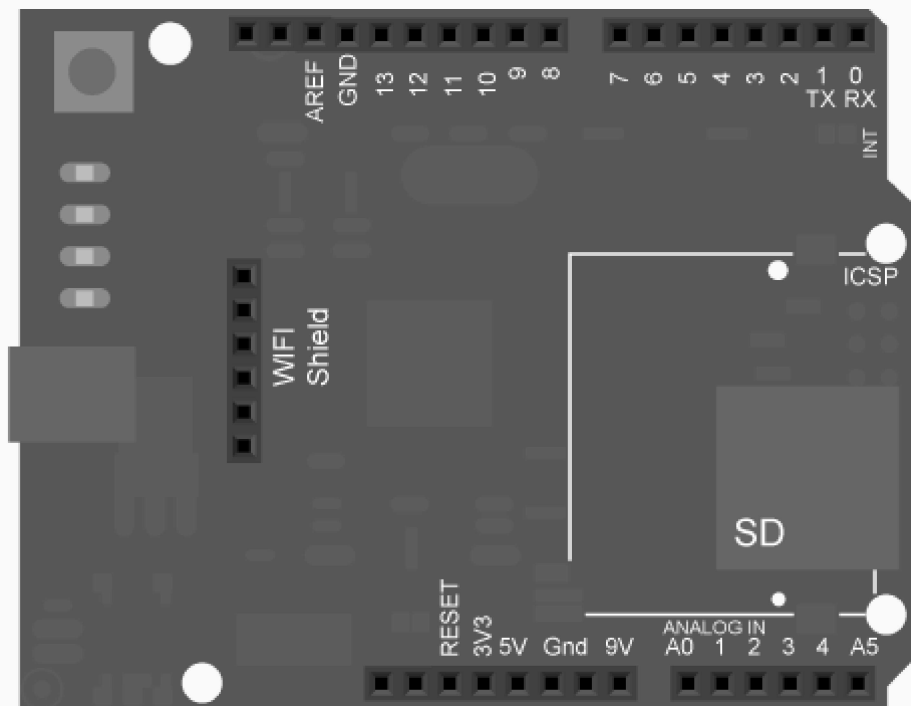
ARDUINO WIFI

Introduction

Using Arduino with WiFi is a great option for connecting the Arduino board wirelessly to the Internet in a few minutes. Connecting a device to a WiFi network is straightforward; no configuration beyond the SSID and password is needed. There are many boards with WiFi connectivity, as it provides an easy setup, without any cable requirement. There are plenty of alternatives for WiFi connectivity, including shields, devices with on-board WiFi, or external modules that can be connected to the microcontroller.

In this documentation, we cover how to connect devices over WiFi by using different approaches, like Arduino Shields, external modules, and devices with embedded WiFi like Arduino Nano 33 IoT, or Arduino MKR WIFI 1010.

Arduino with WiFi Shield



Arduino WiFi shield

This example will allow connecting the Arduino device with the WiFi Shield to the cloud platform in a few lines. The `arduino_secrets.h` file just needs to be modified with the relevant information.

```
#define THINGER_SERIAL_DEBUG
#define THINGER_USE_STATIC_MEMORY
#define THINGER_STATIC_MEMORY_SIZE 512

#include <WiFi.h>
#include <ThingierWifi.h>
#include "arduino_secrets.h"

ThingierWifi thing(USERNAME, DEVICE_ID, DEVICE_CREDENTIAL);

void setup() {
  // open serial for debugging
  Serial.begin(115200);

  // configure wifi network
  thing.add_wifi(SSID, SSID_PASSWORD);

  pinMode(2, OUTPUT);

  // pin control example (i.e. turning on/off a light, a
  relay, etc)
  thing["led"] << digitalPin(2);

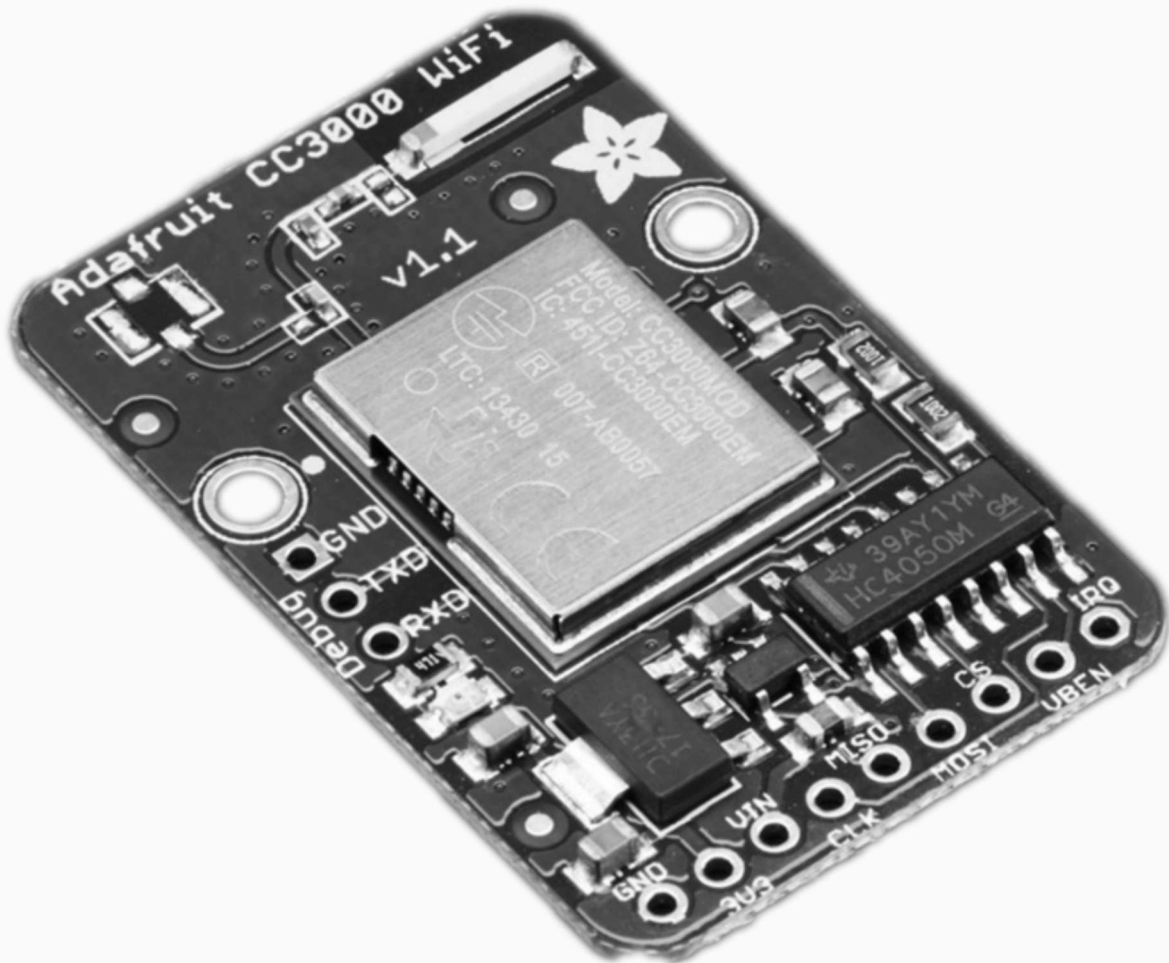
  // resource output example (i.e. reading a sensor value, a
  variable, etc)
  thing["millis"] >> outputValue(millis());

  // more details at http://docs.thinger.io/arduino/
}

void loop() {
  thing.handle();
}
```

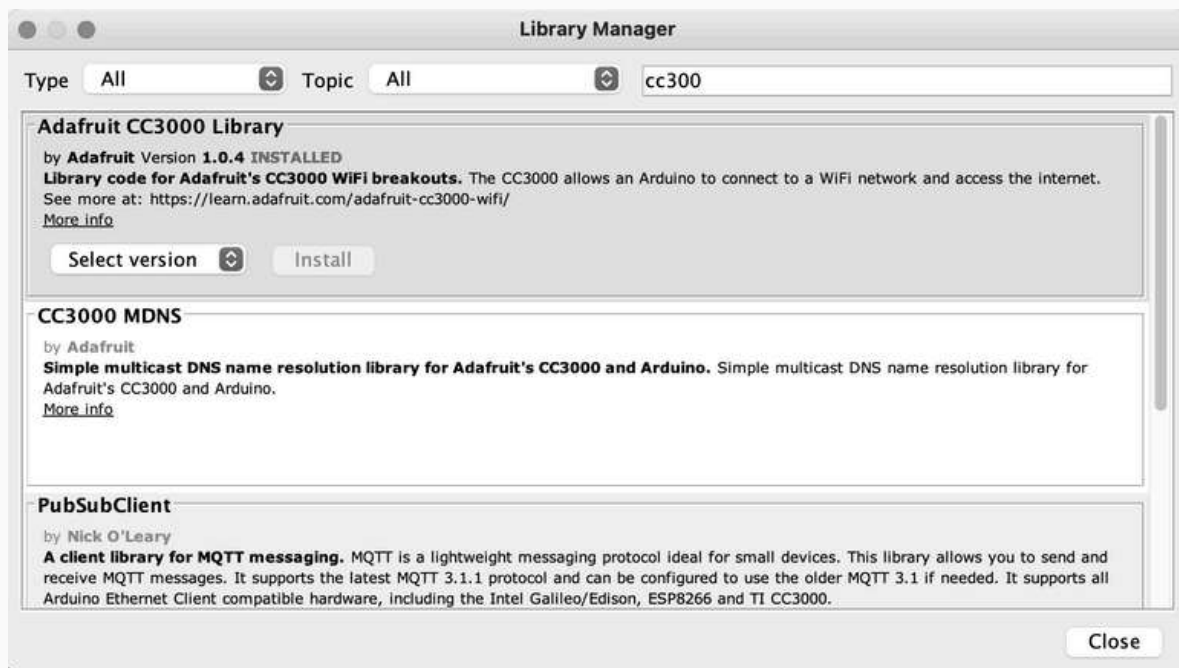
Arduino with CC3000

The CC3000 chip from Texas Instruments was one of the first low-cost WiFi chips that revolutionized the IoT maker ecosystem. In contrast to the other available WiFi alternatives, like the WiFi shield, the CC3000 appeared at a low cost (about 10\$) for its time. It is a powerful chip as it integrates the whole TCP/IP stack and many other protocols. Some vendors, like Adafruit, started to build modules and libraries for integrating this chip with the Arduino ecosystem. Thanks to the libraries provided by Adafruit it is then possible to build a connected device with a few lines of code.



Texas Instruments CC3000 WiFi module

For this module is required to have installed the **Adafruit CC3000 Libraries**, as they are directly used by the Thinger client. Install it directly from the Arduino Library Manager by searching `cc3000`.



Install CC3000 Arduino Libraries

This example will allow connecting the Arduino device with the CC3000 module to the cloud platform in a few lines. The `arduino_secrets.h` file just needs to be modified with the relevant information.

```
#define THINGER_SERIAL_DEBUG

#include <ThingyCC3000.h>
#include "arduino_secrets.h"

ThingyCC3000 thing(USERNAME, DEVICE_ID, DEVICE_CREDENTIAL);

void setup() {
  // open serial for debugging
  Serial.begin(115200);

  // configure wifi network
  thing.add_wifi(SSID, SSID_PASSWORD);

  pinMode(2, OUTPUT);

  // pin control example (i.e. turning on/off a light, a
  relay, etc)
  thing["led"] << digitalPin(2);

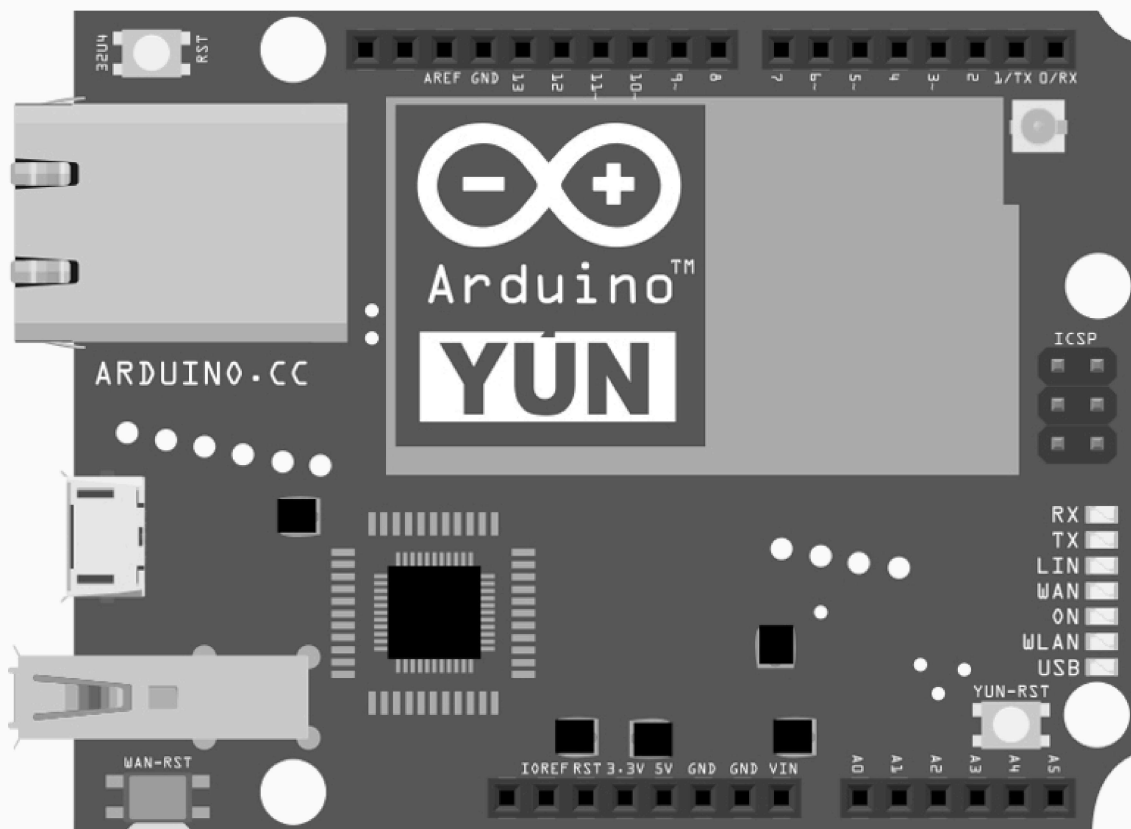
  // resource output example (i.e. reading a sensor value, a
  variable, etc)
  thing["millis"] >> outputValue(millis());

  // more details at http://docs.thingy.io/arduino/
}

void loop() {
  thing.handle();
}
```

Arduino Yun

The Arduino Yún is a microcontroller board based on the ATmega32u4 and the Atheros AR9331. The Atheros processor supports a Linux distribution based on OpenWrt named OpenWrt-Yun. The board has built-in Ethernet and WiFi support, a USB-A port, micro-SD card slot, 20 digital input/output pins (of which 7 can be used as PWM outputs and 12 as analog inputs), a 16 MHz crystal oscillator, a micro USB connection, an ICSP header, and 3 reset buttons. This board lets the programmable ATmega32u4 communicate with the Internet by using the Bridge Library that exposes some functions running in the Linux distribution.



Arduino Yun Board

This example will allow connecting an Arduino Yun to the cloud platform in a few lines using the WiFi interface. The `arduino_secrets.h` file just needs to be modified with the relevant information. Notice that it is not required to configure any network parameters in the code, as this is managed by the running Linux distribution. However, it may be necessary to connect with the Arduino Yun via WiFi to connect it to a local network.

ArduinoYun.ino

arduino_secrets.h

```
#include <Thingeryun.h>
#include "arduino_secrets.h"
```

```
Thingeryun thing(USERNAME, DEVICE_ID, DEVICE_CREDENTIAL);
```



```
void setup() {
```

```
  pinMode(LED_BUILTIN, OUTPUT);
```

```
  // initialize bridge
```

```
  Bridge.begin();
```

```
  // pin control example (i.e. turning on/off a light, a
  relay, etc)
```

```
  thing["led"] << digitalPin(LED_BUILTIN);
```

```
  // resource output example (i.e. reading a sensor value, a
  variable, etc)
```

```
  // more details at http://docs.thinger.io/arduino/
```

```
}
```

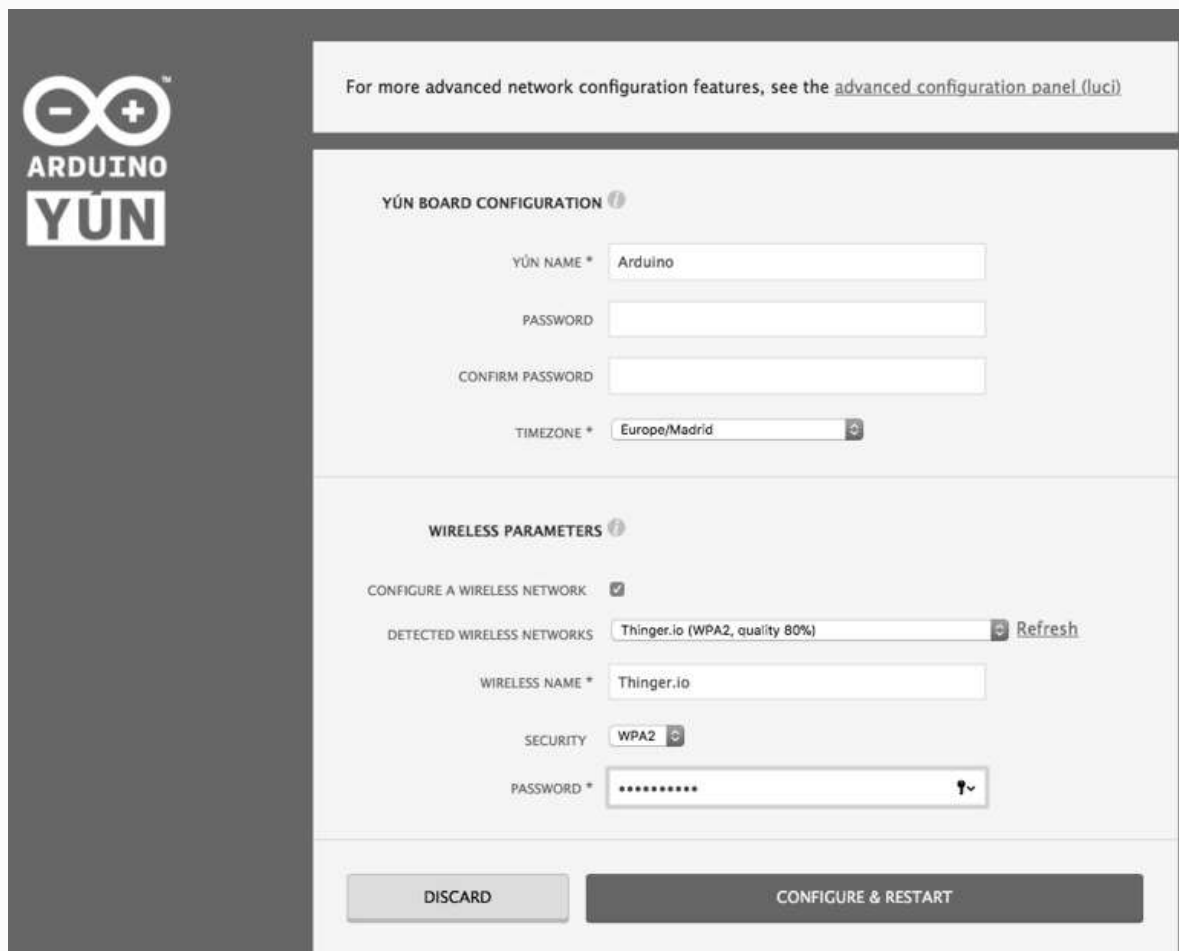
```
void loop() {
```

```
  thing.handle();
```

```
}
```



For using Arduino Yun, the device must be connected to a network with Internet, just via Ethernet or a Wifi connection. It can be configured in the Arduino Yun web configuration.



The screenshot shows the Arduino Yun network configuration web interface. On the left is a dark sidebar with the Arduino Yun logo. The main content area has a light gray background. At the top, a message says: "For more advanced network configuration features, see the [advanced configuration panel \(luci\)](#)". Below this, there are two main sections: "YUN BOARD CONFIGURATION" and "WIRELESS PARAMETERS".

YUN BOARD CONFIGURATION

- YUN NAME *:
- PASSWORD:
- CONFIRM PASSWORD:
- TIMEZONE *:

WIRELESS PARAMETERS

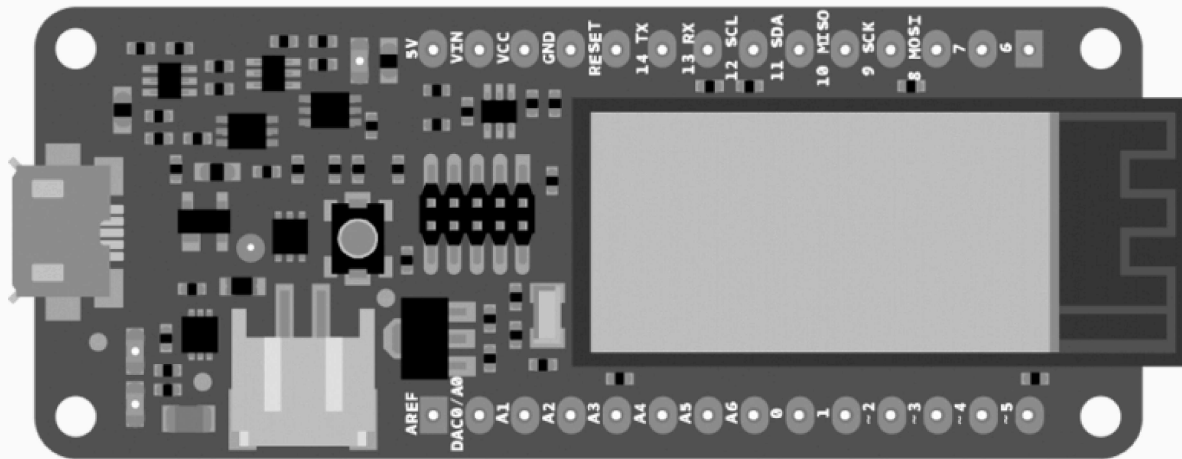
- CONFIGURE A WIRELESS NETWORK: ☒
- DETECTED WIRELESS NETWORKS: [Refresh](#)
- WIRELESS NAME *:
- SECURITY:
- PASSWORD *:

At the bottom, there are two buttons: "DISCARD" and "CONFIGURE & RESTART".

Arduino Yun network configuration

Arduino MKR1000

The Arduino MKR1000 is a microcontroller based on the Atmel ATSAMW25 SoC (System on Chip), which is part of the SmartConnect family of Atmel Wireless devices, specifically designed for IoT projects and devices. A good 32-bit computational power similar to the Zero board, the usual rich set of I/O interfaces, low-power WiFi with a Cryptochip for secure communication, and the ease of use of the Arduino Software (IDE) for code development and programming. All these features make this board the preferred choice for the emerging IoT battery-powered projects in a compact form factor.



This example will allow connecting the MKR1000 device to the cloud platform in a few lines using the WiFi interface. The `arduino_secrets.h` file just needs to be modified with the relevant information.

C++

arduino_secrets.h

```
#define THINGER_SERIAL_DEBUG

#include <ThingierWifi101.h>
#include "arduino_secrets.h"

// cannot connect? Update WiFi101 firmware and add
// iot.thinger.io SSL Certificate
// https://support.arduino.cc/hc/en-us/articles/360016119219

ThingierWifi101 thing(USERNAME, DEVICE_ID, DEVICE_CREDENTIAL);

void setup() {
  // open serial for debugging
  Serial.begin(115200);

  // configure wifi network
  thing.add_wifi(SSID, SSID_PASSWORD);

  pinMode(LED_BUILTIN, OUTPUT);

  // pin control example (i.e. turning on/off a light, a
  // relay, etc)
  thing["led"] << digitalPin(LED_BUILTIN);

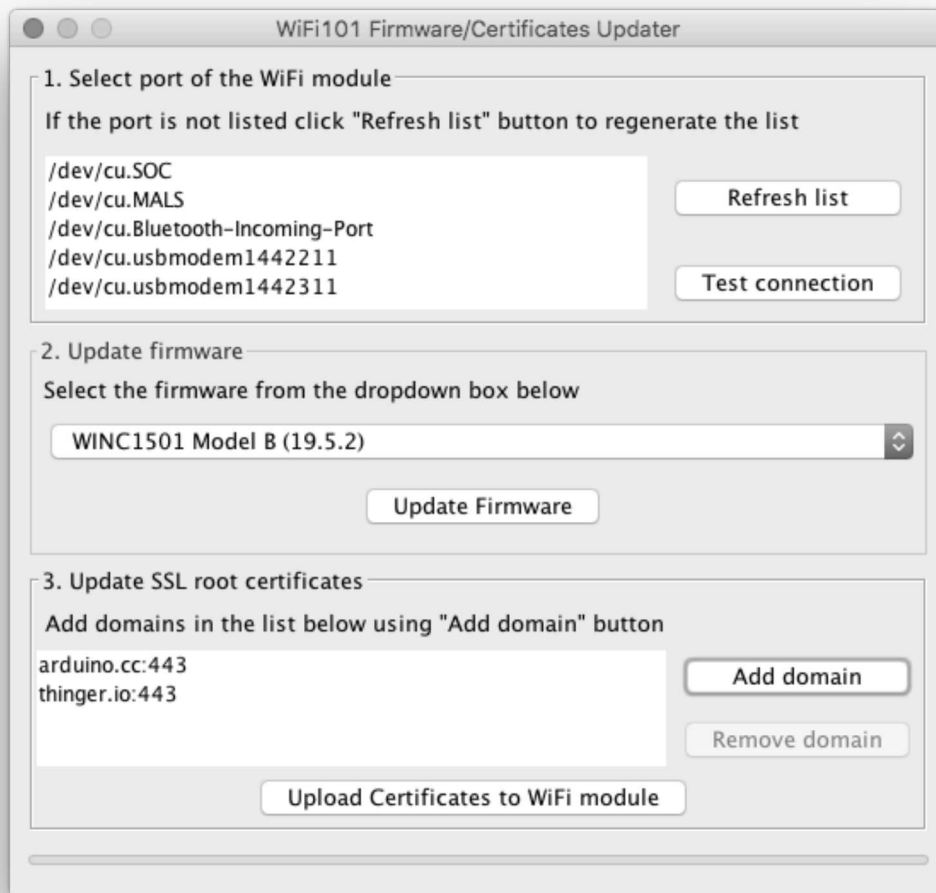
  // resource output example (i.e. reading a sensor value, a
  // variable, etc)
  thing["millis"] >> outputValue(millis());

  // more details at http://docs.thinger.io/arduino/
}

void loop() {
  thing.handle();
}
```



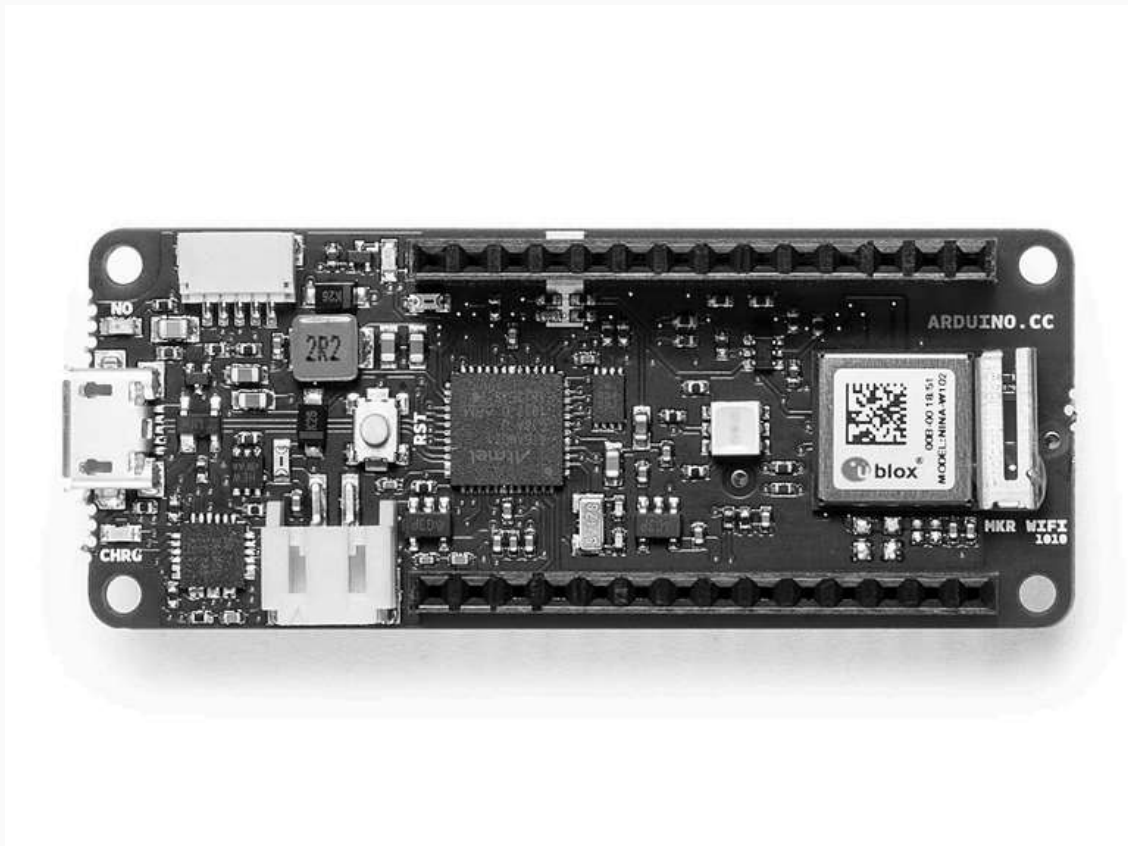
For using MKR1000 over the default TLS/SSL connection, it is required to install the Thinger.io server certificate on the board with the Wifi101 Firmware Updater located in the Tools menu.



WiFi 101 Certificates Updater

Arduino MKR1010

The Arduino MKR WiFi 1010 serves as an accessible entry point for basic IoT and pico-network application design. It is a comprehensive solution for many fundamental IoT application scenarios, whether building a sensor network connected to an office or home router, or creating a BLE device that sends data to a cellphone. The board's primary processor is a low-power Arm® Cortex®-M0 32-bit SAMD21, consistent with other boards in the Arduino MKR family. WiFi and Bluetooth® connectivity are handled by the u-blox NINA-W10 module, a low-power chipset operating in the 2.4GHz range. Additionally, the Microchip® ECC508 crypto chip ensures secure communication. The board also features a battery charger and a directional RGB LED.



Arduino MKR1010

This example will allow connecting the MKR1010 device to the cloud platform in a few lines using the WiFi interface. The `arduino_secrets.h` file just needs to be modified with the relevant information.

- ⚠ The integration with Thingier.io requires downloading an additional library called "Arduino Wi-FiNINA" that allows communicating with the U-BLOX WiFi module.

```
#define THINGER_SERIAL_DEBUG

#include <ThingierWiFiNINA.h>
#include "arduino_secrets.h"

// cannot connect? Update WiFiNINA and add iot.thinger.io SSL
Certificate
// https://support.arduino.cc/hc/en-us/articles/360016119219

ThingierWiFiNINA thing(USERNAME, DEVICE_ID, DEVICE_CREDENTIAL);

void setup() {
  // configure LED_BUILTIN for output
  pinMode(LED_BUILTIN, OUTPUT);

  // open serial for debugging
  Serial.begin(115200);

  // configure wifi network
  thing.add_wifi(SSID, SSID_PASSWORD);

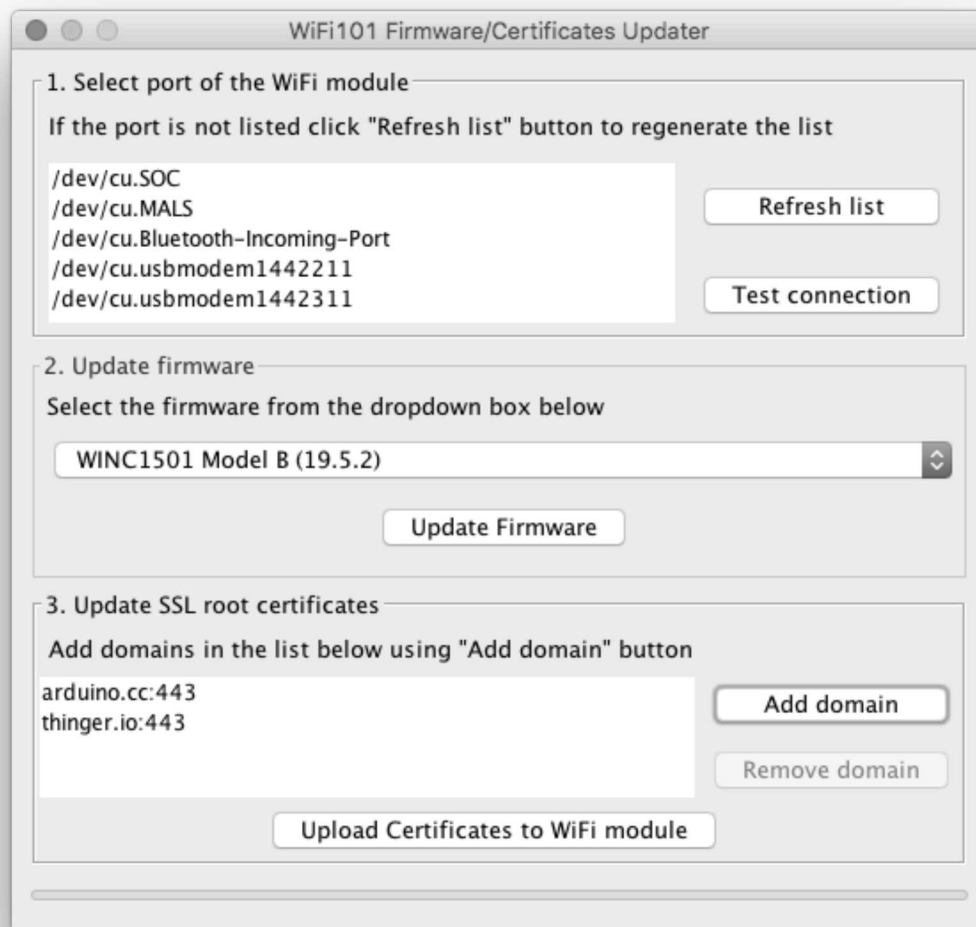
  // pin control example (i.e. turning on/off a light, a
  relay, etc)
  thing["led"] << digitalPin(LED_BUILTIN);

  // resource output example (i.e. reading a sensor value, a
  variable, etc)
  thing["millis"] >> outputValue(millis());

  // more details at http://docs.thinger.io/arduino/
}

void loop() {
  thing.handle();
}
```

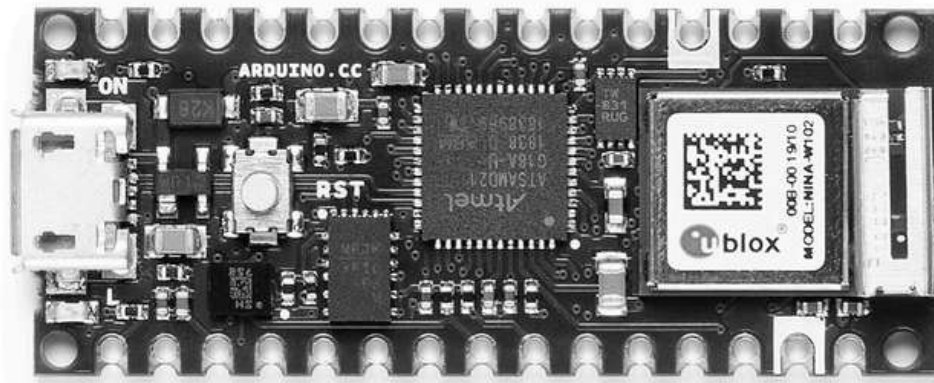
 For using MKR1010 over the default TLS/SSL connection, it is required to install the Thinger.io server certificate in the board with the Wifi101 Firmware Updater located in the Tools menu.



WiFiNINA Certificates Updater

Arduino Nano 33 IoT

In the same iconic size as the Arduino Nano, the Arduino Nano 33 IoT hosts an Arm Cortex-M0+ SAMD21 processor, a WiFi and Bluetooth module based on ESP32, a 6-axis Inertial Measurement Unit (IMU) and a crypto chip which can securely store certificates and pre-shared keys.



- ⓘ The integration with Thingier.io requires downloading an additional library called "Arduino WiFinINA" that allows communicating with the U-BLOX WiFi module.

This example will allow connecting the Arduino Nano 33 IoT device to the cloud platform in a few lines using the WiFi interface. The `arduino_secrets.h` file just needs to be modified with the relevant information.

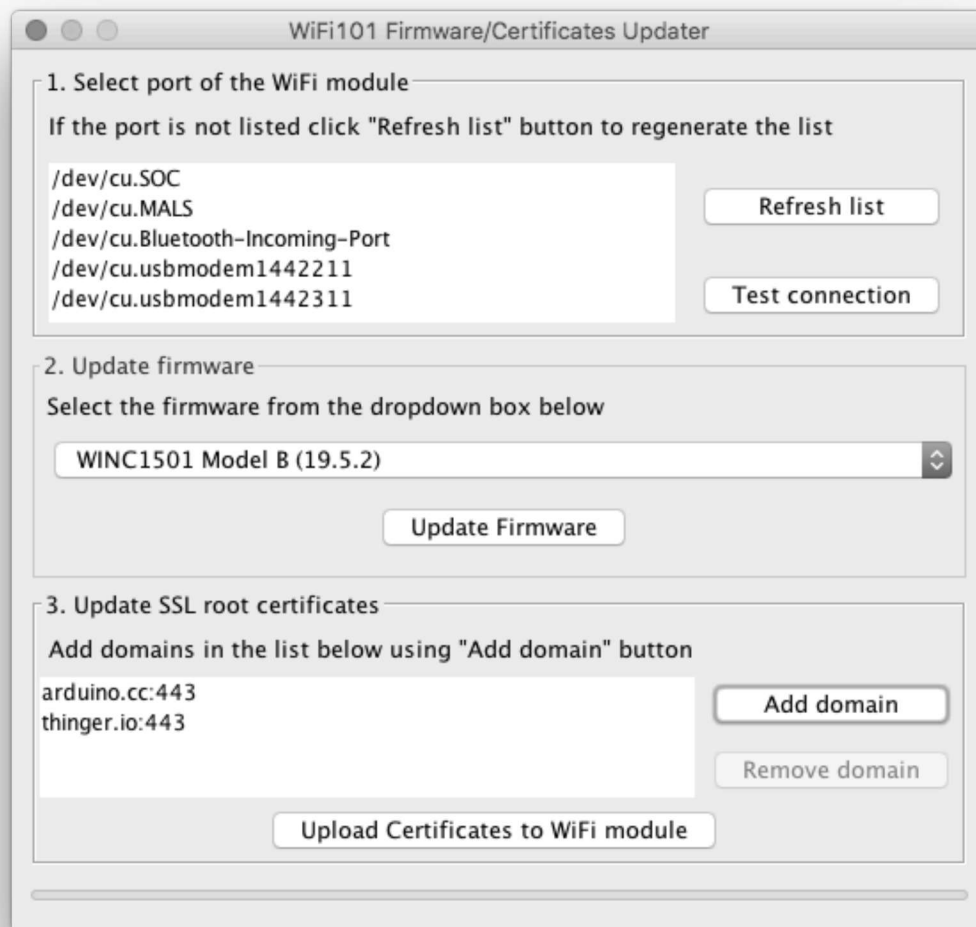
ArduinoNano33IoT.ino

arduino_secrets.h

```
#define USERNAME "your_user_name"
#define DEVICE_ID "your_device_id"
#define DEVICE_CREDENTIAL "your_device_credential"

#define SSID "your_wifi_ssid"
#define SSID_PASSWORD "your_wifi_ssid_password"
```

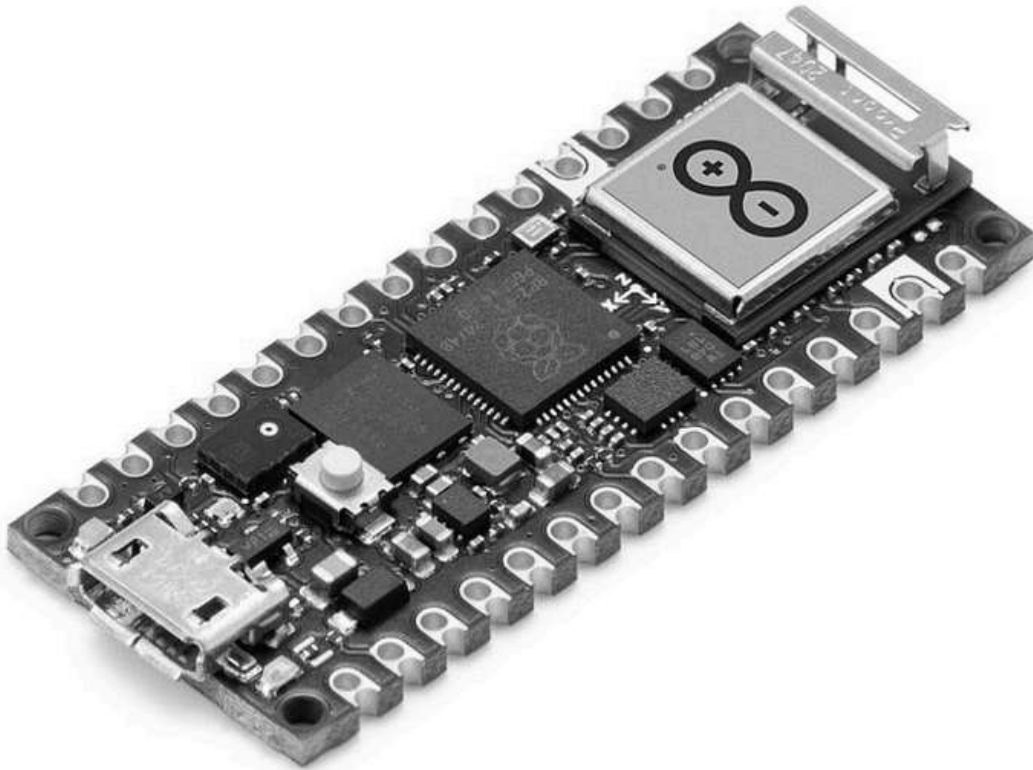
- ⓘ For using Arduino 33 IoT over the default TLS/SSL connection, it is required to install the Thingier.io server certificate in the board with the Wifi101 Firmware Updater located in the Tools menu.



WiFiNINA Certificates Updater

Arduino Nano RP2040

The brain of the board is the Raspberry Pi RP2040 silicon, a dual-core Arm Cortex M0+ running at 133MHz. It has 264KB of SRAM, and the 16MB of flash memory is off-chip to give extra storage. But what's really exciting is the onboard connectivity options. The hugely popular and highly adaptable u-blox NINA-W102 radio module is on there to make this a true IoT champion. It has on-board, built-in sensors to turn builds into powerhouse projects as well. Microphone and motion sensing add a depth of possibilities that's almost impossible to find in a board of this size. The Arduino Nano RP2040 Connect is the premium choice for RP2040 devices and the perfect option for upgrading projects and unlocking the potential of new ones.



Arduino Nano RP2040

- ⚠ The integration with Thingier.io requires downloading an additional library called "Arduino WiFININA" that allows communicating with the U-BLOX WiFi module.

This example will allow connecting the Arduino Nano RP2040 device to the cloud platform in a few lines using the WiFi interface. The `arduino_secrets.h` file just needs to be modified with the relevant information.

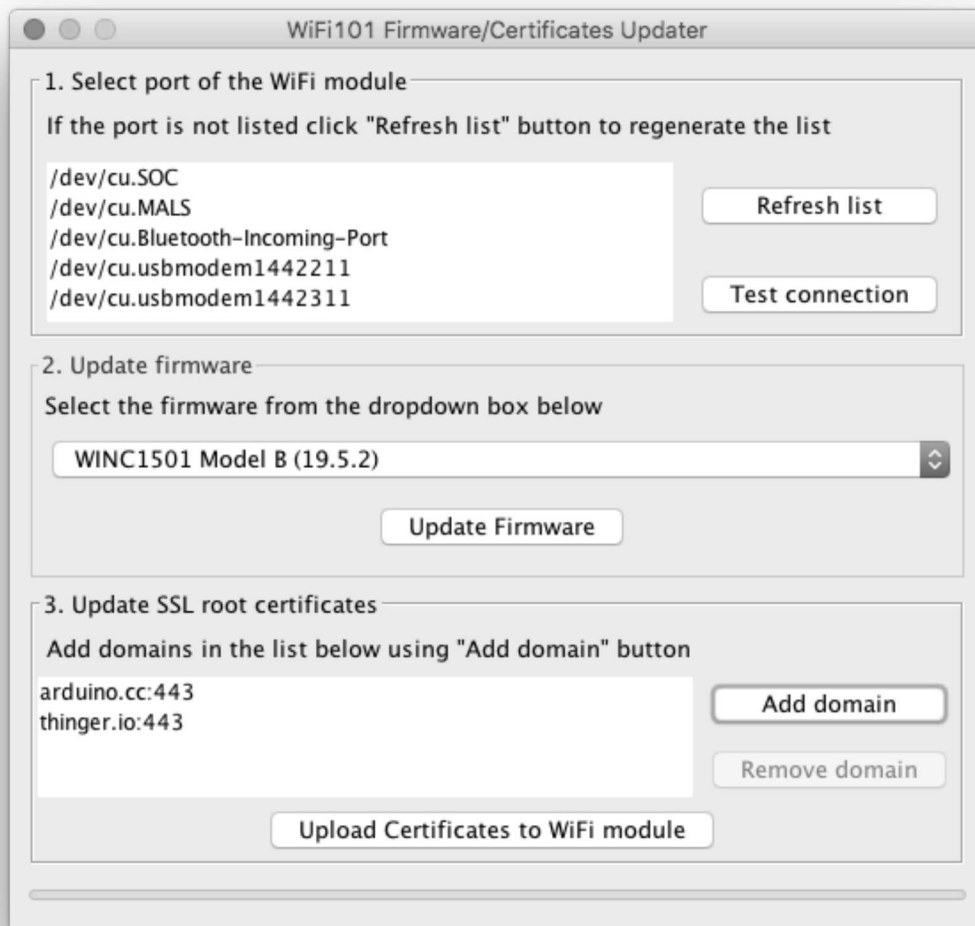
ArduinoNanoRP2040.ino

arduino_secrets.h

```
#define USERNAME "your_user_name"
#define DEVICE_ID "your_device_id"
#define DEVICE_CREDENTIAL "your_device_credential"

#define SSID "your_wifi_ssid"
#define SSID_PASSWORD "your_wifi_ssid_password"
```

-  For using Arduino Nano RP2040 over the default TLS/SSL connection, it is required to install the Thingier.io server certificate in the board with the Wifi101 Firmware Updater located in the Tools menu.



The screenshot shows a macOS-style window titled "WiFi101 Firmware/Certificates Updater". It contains three main sections:

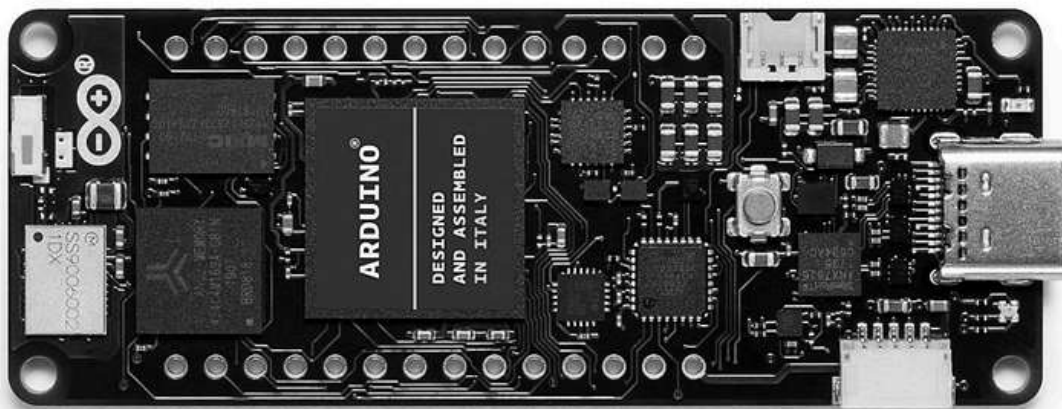
- 1. Select port of the WiFi module**: Includes a text box with a list of ports: `/dev/cu.SOC`, `/dev/cu.MALS`, `/dev/cu.Bluetooth-Incoming-Port`, `/dev/cu.usbmodem1442211`, and `/dev/cu.usbmodem1442311`. To the right are "Refresh list" and "Test connection" buttons.
- 2. Update firmware**: Includes a dropdown menu showing "WINC1501 Model B (19.5.2)" and an "Update Firmware" button.
- 3. Update SSL root certificates**: Includes a text box with "arduino.cc:443" and "thingier.io:443", "Add domain" and "Remove domain" buttons, and an "Upload Certificates to WiFi module" button.

WiFiNINA Certificates Updater

Arduino Portenta H7

Portenta H7 simultaneously runs high-level code along with real-time tasks. The design includes two processors that can run tasks in parallel. For example, it is possible to execute Arduino compiled code along with MicroPython code, and have both cores communicate with one another. The Portenta functionality is two-fold, it can either be running like any other embedded microcontroller board or as the main processor of an embedded computer.

H7's main processor is the dual-core STM32H747, including a Cortex® M7 running at 480 MHz and a Cortex® M4 running at 240 MHz. The two cores communicate via a *Remote Procedure Call* mechanism that allows calling functions on the other processor seamlessly.



Arduino Portenta H7

```

#define THINGER_SERIAL_DEBUG

#include <Thingermbed.h>
#include "arduino_secrets.h"

Thingermbed thing(USERNAME, DEVICE_ID, DEVICE_CREDENTIAL);

void setup() {
  // configure LED_BUILTIN for output
  pinMode(LED_BUILTIN, OUTPUT);

  // open serial for debugging
  Serial.begin(115200);

  // configure wifi network
  thing.add_wifi(SSID, SSID_PASSWORD);

  // pin control example (i.e. turning on/off a light, a
  relay, etc)
  thing["led"] << digitalPin(LED_BUILTIN);

  // resource output example (i.e. reading a sensor value, a
  variable, etc)
  thing["millis"] >> outputValue(millis());

  // more details at http://docs.thinger.io/arduino/
}

void loop() {
  thing.handle();
}

```



In case of problems when connecting over secure TLS connections, try updating the WiFi firmware by flashing the WiFiFirmwareUpdater example sketch.



Arduino Opta Wifi

The Arduino Opta is designed for industrial automation, offering robust performance and reliability. It features a dual-core STM32H747 microcontroller, which includes a Cortex® M7 running at 480 MHz and a Cortex® M4 running at 240 MHz. This configuration enables Opta to handle complex real-time tasks and high-level code execution concurrently.

With its versatile architecture, the Opta supports running Arduino sketches alongside MicroPython, allowing developers to leverage the strengths of both programming environments. The dual-core setup facilitates inter-core communication via Remote Procedure Call, ensuring smooth and efficient coordination between the two processors. This capability makes the Arduino Opta ideal for advanced automation systems, where precise control and rapid response are crucial.

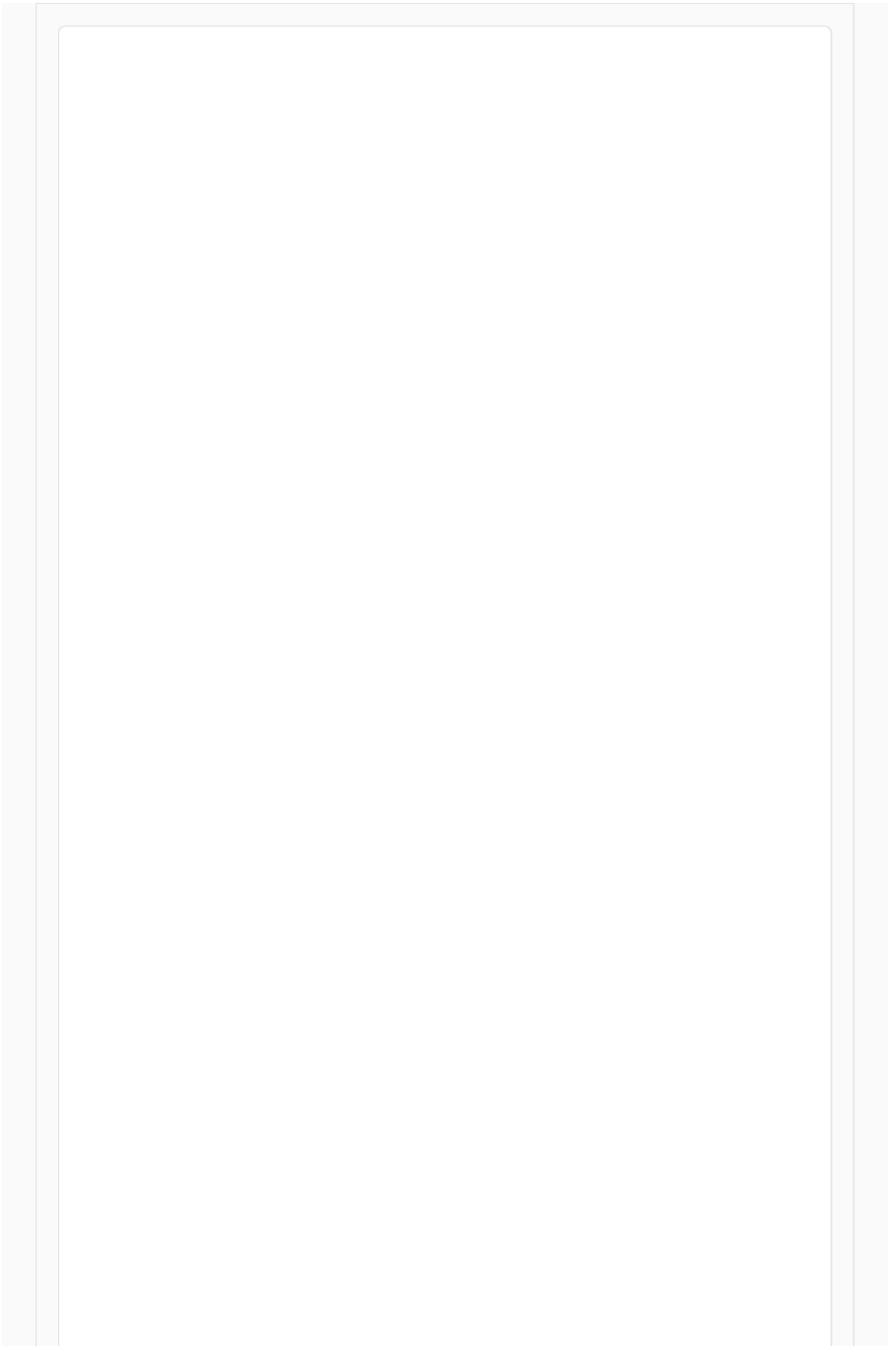
Additionally, the Arduino Opta is equipped with industrial-grade features, such as enhanced I/O capabilities and robust connectivity options. It can be seamlessly integrated into existing industrial networks, providing a reliable solution for monitoring and control applications. Whether used as a standalone microcontroller or as part of a larger embedded system, the Opta's performance and versatility make it a valuable asset in any industrial setting.



Arduino Opta Wifi

ArduinoOptaWifi.ino

arduino_secrets.h



```

#define THINGER_SERIAL_DEBUG

#include <ThingyMbed.h>
#include <ThingyPortentaOTA.h>
#include "arduino_secrets.h"

ThingyMbed thing(USERNAME, DEVICE_ID, DEVICE_CREDENTIAL);
ThingyPortentaOTA ota(thing);

void setup() {
    // open serial for debugging
    Serial.begin(115200);

    // configure leds for output
    pinMode(LED_D0, OUTPUT);
    pinMode(LED_D1, OUTPUT);
    pinMode(LED_D2, OUTPUT);
    pinMode(LED_D3, OUTPUT);
    pinMode(LED_R, OUTPUT);
    pinMode(LED_BUILTIN, OUTPUT);

    // configure relays for output
    pinMode(D0, OUTPUT);
    pinMode(D1, OUTPUT);
    pinMode(D2, OUTPUT);
    pinMode(D3, OUTPUT);

    // example for controlling relays and status LED
    thing["relay_d0"] << [](pson& in){
        if(in.is_empty()){
            in = (bool) digitalRead(D0);
        }else{
            digitalWrite(D0, in ? HIGH : LOW);
            digitalWrite(LED_D0, in ? HIGH : LOW);
        }
    };

    thing["relay_d1"] << [](pson& in){
        if(in.is_empty()){
            in = (bool) digitalRead(D1);
        }else{
            digitalWrite(D1, in ? HIGH : LOW);
            digitalWrite(LED_D1, in ? HIGH : LOW);
        }
    };

    thing["relay_d2"] << [](pson& in){
        if(in.is_empty()){

```

```

        in = (bool) digitalRead(D2);
    }else{
        digitalWrite(D2, in ? HIGH : LOW);
        digitalWrite(LED_D2, in ? HIGH : LOW);
    }
};

thing["relay_d3"] << [](pson& in){
    if(in.is_empty()){
        in = (bool) digitalRead(D3);
    }else{
        digitalWrite(D3, in ? HIGH : LOW);
        digitalWrite(LED_D3, in ? HIGH : LOW);
    }
};

// example for controlling the LED
thing["led"] << digitalPin(LED_BUILTIN);
thing["led_r"] << digitalPin(LED_R);

// resource output example (i.e. reading a sensor value, a
variable, etc)
thing["millis"] >> outputValue(millis());

// start thinger on its own task
thing.start();

// more details at http://docs.thinger.io/arduino/
}

void loop() {
    // use loop as in normal Arduino Sketch
    // use thing.lock() thing.unlock() when using/modifying
    variables exposed on thinger resources
    delay(1000);
}

```

- ☑ In case of problems when connecting over secure TLS connections, try updating the WiFi firmware by flashing the WiFiFirmwareUpdater example sketch.



Arduino Uno WiFi Rev2

The Arduino Uno WiFi is functionally the same as the Arduino Uno Rev3, but with the addition of WiFi and some other enhancements. It incorporates a brand new 8-bit microprocessor from Microchip and has an onboard IMU (Inertial Measurement Unit). The WiFi Module is a self-contained SoC with an integrated TCP/IP protocol stack that can provide access to a WiFi network or act as an access point.

ⓘ The integration with Thingier.io requires downloading an additional library called "Arduino WiFinINA" that allows communicating with the U-BLOX WiFi module.

This example will allow connecting the Arduino Uno WiFi Rev2 device to the cloud platform in a few lines using the WiFi interface. The `arduino_secrets.h` file just needs to be modified with the relevant information.

```
#define THINGER_SERIAL_DEBUG

#include <ThingierWiFiNINA.h>
#include "arduino_secrets.h"

// cannot connect? Update WiFiNINA and add iot.thinger.io SSL
Certificate
// https://support.arduino.cc/hc/en-us/articles/360016119219

ThingierWiFiNINA thing(USERNAME, DEVICE_ID, DEVICE_CREDENTIAL);

void setup() {
  // configure LED_BUILTIN for output
  pinMode(LED_BUILTIN, OUTPUT);

  // open serial for debugging
  Serial.begin(115200);

  // configure wifi network
  thing.add_wifi(SSID, SSID_PASSWORD);

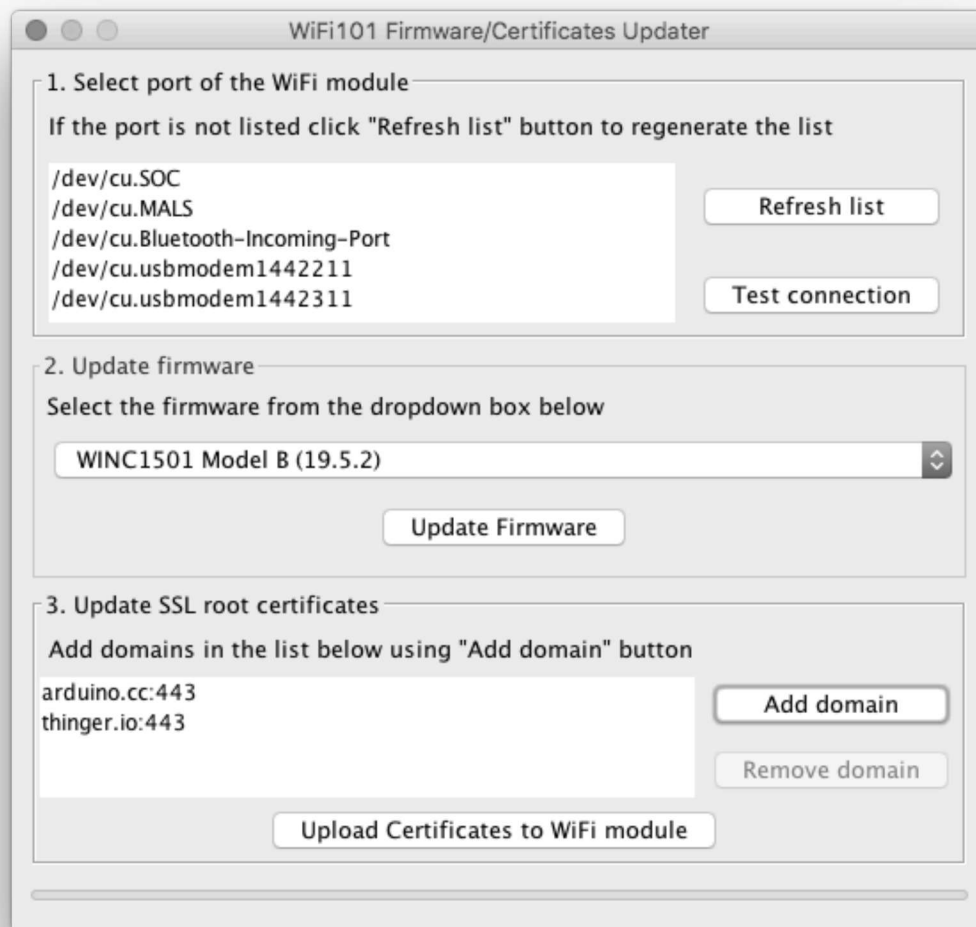
  // pin control example (i.e. turning on/off a light, a
  relay, etc)
  thing["led"] << digitalPin(LED_BUILTIN);

  // resource output example (i.e. reading a sensor value, a
  variable, etc)
  thing["millis"] >> outputValue(millis());

  // more details at http://docs.thinger.io/arduino/
}

void loop() {
  thing.handle();
}
```

-  For using this board with the default TLS/SSL connection, it is required to install the Thinger.io server certificate in the board with the Wifi101 Firmware Updater located in the Tools menu.



Previous
ARDUINO ETHERNET

Next
ARDUINO GSM

Last updated 5 months ago

Was this helpful?

